

개인 포트폴리오

바람의 나라

C++ 콘솔 환경에서 구현한 2D MMORPG 모작



박 찬 욱

2016. 12. 25 ~ 2016. 12. 30

1. 개요

작품 소개

넥슨의 2D MMORPG **바람의 나라**를 **C++ 콘솔 프로그램**으로 모작한 작품이다. 게임 엔진이나 그래픽 라이브러리를 일절 쓰지 않고, Win32 API 와 GDI 만으로 타일맵 · 스프라이트 애니메이션 · 전투 · 인벤토리 · 상점 · 대화형 퀘스트까지 하나의 실행 파일에 담았다. 학부 재학 중 **6일** 동안 혼자 만들었다.

콘솔 창은 본래 문자만 출력하는 환경이다. 이 작품의 출발점은 **그 제약을 우회할 방법을 찾아낸 것으로**, 콘솔 창의 윈도우 핸들을 직접 얻어 GDI 로 비트맵을 그려 넣는 방식을 사용했다. 덕분에 콘솔 프로그램이면서도 원작과 거의 같은 화면을 재현할 수 있었다.

개발 환경	Visual Studio 2015 (v140) · Win32 Console Application · C++
사용 기술	Win32 API, GDI / msimg32 (TransparentBlt), winmm (PlaySound)
외부 의존성	없음 — 게임 엔진 · 그래픽 · 물리 라이브러리 미사용
개발 기간	6일 (2016. 12. 25 ~ 12. 30), 1인 개발
소스 규모	70개 파일 / 36개 클래스 / 약 3,780줄
리소스	BMP 스프라이트 76장, 애니메이션 60개, 이미지 키 68개
구현 범위	타일맵 3종, 몬스터 3종, NPC 2종, 아이템 3종, 7단계 퀘스트 체인

구현 기능

- 타일 단위 이동과 픽셀 단위 보간을 함께 쓰는 캐릭터 이동
- 타일맵에 충돌을 기록하는 예약 방식 충돌 처리
- Port 오브젝트를 통한 맵 간 이동 (3개 맵, 양방향)
- 상태 · 방향에 따라 자동 선택되는 16슬롯 스프라이트 애니메이션
- 플레이어 / 몬스터 유한 상태 머신 (배회 · 추격 · 공격)
- 대화 노드 그래프로 구현한 수집 · 사냥 퀘스트
- 인벤토리, 아이템 사용 · 획득, 상점 매매, 사망 후 부활



사냥터 '수상한 숲'. 다람쥐 · 토끼가 각자 배회하고, 우측에 인벤토리와 스탯 UI 가 표시된다.

2. 콘솔 창에 그래픽을 출력하기

콘솔은 문자 단위 출력만 제공한다. 색을 바꾸거나 커서를 옮기는 정도는 되지만, 임의의 좌표에 그림을 찍는 인터페이스는 없다. 하지만 콘솔 창도 결국 하나의 윈도우다. 창이 있다면 그 창을 그릴 수 있는 장치 컨텍스트(DC)도 존재한다는 뜻이다.

2.1 창 핸들을 얻어 GDI 로 그리기

`GetConsoleWindow()` 로 콘솔 창의 핸들을 얻고, `GetDC()` 로 그 창의 DC 를 확보하면 그 시점부터는 일반 Win32 GUI 프로그램과 다를 게 없다. 콘솔 API 대신 GDI 함수로 비트맵을 그려 넣으면 된다.

```
system ("mode con:cols=53 lines=18"); // 창 크기를 문자 격자로 지정

CONSOLE_CURSOR_INFO ConsoleCursor;
ConsoleCursor.bVisible = FALSE; // 깜빡이는 커서 제거
SetConsoleCursorInfo (GetStdHandle (STD_OUTPUT_HANDLE), &ConsoleCursor);

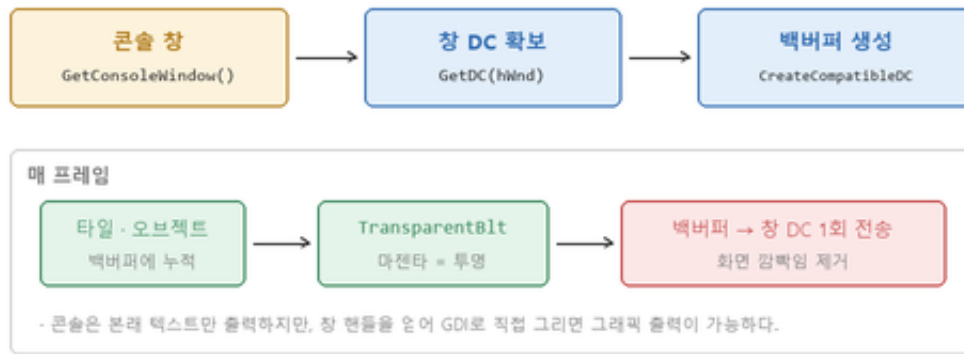
gHwnd = GetConsoleWindow (); // 콘솔 창의 윈도우 핸들
gHDC = GetDC (gHwnd); // 그 창에 그릴 수 있는 DC
```

창 크기는 `mode con` 명령으로 문자 격자 수를 지정해 맞췄고, 텍스트 커서는 게임 화면 위에서 깜빡이면 눈에 거슬리므로 감췄다.

2.2 더블 버퍼링

타일 · 오브젝트 · UI 를 창 DC 에 직접 하나씩 그리면, 그리는 과정이 그대로 화면에 보여 심하게 깜빡인다. 그래서 창과 호환되는 메모리 DC 와 비트맵을 만들어 백버퍼로 쓰고, 한 프레임의 모든 그리기를 백버퍼에 누적한 뒤 마지막에 한 번만 창 DC 로 옮긴다.

그림 1. 콘솔 창을 그래픽 서피스로 사용하는 렌더링 파이프라인



2.3 투명 처리

BMP 는 알파 채널이 없다. 캐릭터를 사각형 그대로 찍으면 배경까지 함께 덮어버린다. 그래서 스프라이트의 배경을 게임에 등장하지 않는 **마젠타(255, 0, 255)**로 칠해 두고, msimg32 의 `TransparentBlt` 에 그 색을 컬러키로 넘겨 해당 픽셀을 건너뛰게 했다.

```
TransparentBlt (_hBackDC,
    x + image->GetOffsetX (), y + image->GetOffsetY (),
    image->GetWidth (), image->GetHeight (),
    _hMemDC, 0, 0, image->GetWidth (), image->GetHeight (),
    RGB (255, 0, 255)); // 이 색은 찍지 않는다
```

그리기 순서는 **뒤에서 앞으로** 쌓는 방식을 그대로 따랐다. 타일 → 아이템 → 몬스터 → 플레이어 → NPC → 장식물 → UI → 대화창 순으로 그려서, 깊이 정렬 없이도 캐릭터가 타일 위에 서고 대화창이 항상 맨 위에 오도록 했다.

3. 리소스 확보 — 원본 화면에서 스프라이트 추출

모작이므로 사용할 그래픽 리소스가 따로 없었다. 원작을 실행해 화면을 캡처하고, 캐릭터 · 몬스터 · 타일 · UI 프레임을 **한 장씩 직접 잘라내** BMP 로 정리했다. 아틀라스 도구나 스프라이트 에디터 없이 만든 리소스라, 스프라이트마다 기준점이 제각각인 문제가 있었다.

그림 7. 원본 게임 스크린샷에서 스프라이트를 직접 추출



- 게임 엔진의 스프라이트 에디터나 아틀라스 도구 없이, 캡처 화면에서 프레임을 한 장씩 잘라 BMP 로 정리했다.
- 알파 채널이 없는 BMP 이므로 배경을 마젠타로 칠하고, `TransparentBlt` 의 컬러키로 지정해 투명 처리했다.
- 스프라이트마다 기준점이 다르므로 등록 시점에 오프셋을 함께 넣어, 24px 격자 위에 발끝이 맞도록 보정했다.
- 총 76 장의 BMP, 애니메이션 60 개, 이미지 키 68 개를 등록해 사용했다.

3.1 이미지별 오프셋 보정

타일 한 칸은 24px 인데 캐릭터 스프라이트는 그보다 훨씬 크고, 잘라낸 여백도 이미지마다 다르다. 이걸 그리기 코드에서 매번 보정하면 호출부가 지저분해진다. 그래서 **이미지를 등록하는 시점에 오프셋을 함께 저장**하고, 그리기 함수가 좌표에 그 오프셋을 자동으로 더하도록 했다. 호출부는 타일 좌표만 넘기면 된다.

```
// 키, 파일 경로, offsetX, offsetY, 종류
AddImage ("Player Front Idle", L"Image\\Player\\Player Front Idle.bmp", 4, -20, ...);
AddImage ("Virgin Ghost Front Attack", L"...Virgin Ghost Front Attack.bmp", -11, -22, ...);
AddImage ("Tile 0", L"Image\\Tile\\Tile 0.bmp", 0, 0, ...);
```

설계 의도

- 스프라이트의 **발끝**이 타일 격자에 맞도록 Y 오프셋을 음수로 밀어, 키가 큰 캐릭터도 자기 타일 위에 서 있게 보인다.
- 공격 모션처럼 좌우로 크게 뻗는 프레임은 X 오프셋을 음수로 줘서, 무기가 옆 타일로 자연스럽게 넘어가게 했다.
- 리소스의 불규칙함을 **데이터로 흡수**해 로직에서 분리한 셈이다.

3.2 애니메이션

애니메이션은 이미지 키의 목록과 프레임 간 지연 시간만 들고 있는 단순한 구조다. 다만 `IsEnded()` 플래그를 뒤서, 한 사이클이 끝났는지를 외부에서 알 수 있게 했다. 이 플래그가 **상태 머신의 전이 조건**으로 쓰인다 — 공격 상태는 공격 애니메이션이 끝나는 순간에 종료되고, 몬스터의 타격 판정도 그 시점에 일어난다. 애니메이션 길이와 게임 로직의 타이밍이 자동으로 일치하게 되는 구조다.

4. 타일 단위 이동과 픽셀 단위 보간

원작은 타일 기반 게임이지만 캐릭터가 타일 사이를 **미끄러지듯** 이동한다. 타일 단위로 순간이동하면 로직은 간단하지만 움직임이 뚝뚝 끊기고, 픽셀 단위로만 다루면 충돌과 상호작용 판정이 복잡해진다. 두 좌표계를 동시에 유지하는 방식으로 풀었다.

그림 2. 이중 좌표계와 타일 단위 이동



격자 단위로 판정하고 픽셀 단위로 보여주기 때문에, 로직은 타일맵처럼 단순하면서 화면은 부드럽게 움직인다.
 타일 정중앙에 도달한 프레임에서만 맵 이동 · 다음 입력 · 상태 전환을 판정하므로, 이동 중간 상태가 끼여들 여지가 없다.
 방향키를 계속 누르고 있으면 도착 프레임에서 곧바로 다음 타일을 예약해 연속 이동이 끊기지 않는다.

4.1 두 좌표계

모든 오브젝트는 `_worldX/_worldY` (타일 격자 좌표) 와 `_localX/_localY` (픽셀 좌표) 를 함께 갖는다. 타일 좌표를 설정하면 픽셀 좌표가 따라오고, 이동 중에는 반대로 픽셀 좌표에서 타일 좌표를 역산한다.

```
void SetWorldX (int x) { _worldX = x; _localX = x * 24; } // 타일 → 픽셀

void Object::UpdateWorldToLocalXY () // 픽셀 → 타일
{
    _worldX = _localX / 24;
    _worldY = _localY / 24;
}
```

4.2 도착 판정과 연속 이동

이동은 프레임마다 픽셀 좌표를 1 씩 옮기는 것으로 끝이다. 중요한 건 **언제 도착으로 볼지**인데, 픽셀 좌표가 타일 크기로 정확히 나누어떨어지는 순간이 곧 타일 정중앙이다.

```
_localX += _dirX;
_localY += _dirY;

if (_localX % 24 == 0 && _localY % 24 == 0) // 타일 정중앙에 도달
{
    // 이 프레임에서만 맵 이동 · 다음 입력 · 상태 전환을 판정한다
    Port* port = currentMap->CheckPort (_worldX, _worldY);
    ...
    if (방향키가 눌러 있으면) KeyInputWithMoveState (...); // 곧바로 다음 타일 예약
    else state = IDLE;
}
```

이 구조에서 얻은 것

- 판정 지점이 한 곳으로 모인다. 맵 이동 · 포트 진입 · 입력 재판정 · 상태 전환이 모두 '타일 정중앙에 도달한 프레임'에서만 일어나므로, 이동 중간 상태가 다른 로직에 끼어들 여지가 없다.
- 연속 이동이 끊기지 않는다. 도착 프레임에서 방향키가 여전히 눌러 있으면 그 자리에서 다음 타일을 예약하고 이동을 이어받는다. 타일마다 멈칫하는 느낌이 나지 않는다.

· 로직은 격자, 표현은 픽셀. 충돌 · 상호작용은 정수 타일 좌표로 단순하게 다루면서, 화면에는 부드러운 움직임이 나온다.

5. 타일맵에 충돌을 기록하는 예약 방식 충돌 처리

오브젝트끼리 짝지어 겹침을 검사하면 개체 수가 늘어날 때 비용이 제곱으로 커진다. 타일 기반 게임이라면 굳이 그럴 필요가 없다. 한 타일에는 하나만 서 있을 수 있으므로, **충돌 정보를 오브젝트가 아니라 타일이 들고 있게** 하면 갈 곳의 타일 하나만 보면 된다.

그림 3. 타일에 충돌을 기록하는 예약 방식 충돌 처리



P 가 이동을 시작하는 순간 목표 타일을 미리 COLLISION 으로 예약한다. 아직 P 는 도착하지 않았지만 몬스터 M 은 그 타일을 이미 막힌 곳으로 보고 진입을 포기한다 — 이동 중 겹침이 원천적으로 발생하지 않는다.

5.1 이동 시작 시점에 목표 타일을 예약한다

여기서 문제가 하나 생긴다. 픽셀 단위로 이동하는 동안 캐릭터는 **두 타일 사이에 걸쳐** 있다. 아직 목표 타일에 도착하지 않았으니, 그 사이에 다른 몬스터가 같은 타일로 들어와 겹칠 수 있다. 그래서 이동을 **시작하는 순간** 목표 타일을 미리 충돌 상태로 예약한다.

```
moveTile = currentMap->GetTile (_worldX + dirX, _worldY + dirY);

if (moveTile && moveTile->GetCollider () != Object::COLLISION)
{
    state = MOVE;
    // 이동할 지점에 다른 오브젝트가 들어오면 안되므로 충돌 지역으로 한다
    moveTile->SetCollider (Object::COLLISION); // 예약
}
else
{
    state = IDLE; // 막혀 있으면 제자리
}
```

5.2 떠난 타일은 자동으로 되돌린다

예약만 하고 해제를 빠뜨리면 지나온 자리마다 **유령 충돌**이 남아 맵이 점점 막힌다. 모든 오브젝트가 매 프레임 자기가 밟고 있는 타일을 확인하고, 타일이 바뀌었으면 이전 타일을 정상으로 되돌리는 공통 루틴을 **Object** 에 뒀다.

```
void Object::StepTileColliderUpdate ()
{
    // 밟고 있는 타일이 바뀌었을 경우
    if (_stepTile != NULL && _stepTile != 현재타일)
        _stepTile->SetCollider (Tile::NORMAL); // 이전 타일 해제

    _stepTile = 현재타일;
    _stepTile->SetCollider (_collider); // 지금 타일에 자기 속성 기록
}
```

맵을 이동할 때도 같은 이유로 예약을 먼저 풀어줘야 한다. 떠나는 맵의 타일에 충돌이 남으면 다시 돌아왔을 때 지나갈 수 없는 칸이 되기 때문이다. 실제로 겪은 버그였고, 포트 처리 코드에서 전환 직전에 명시적으로 해제하도록 했다.

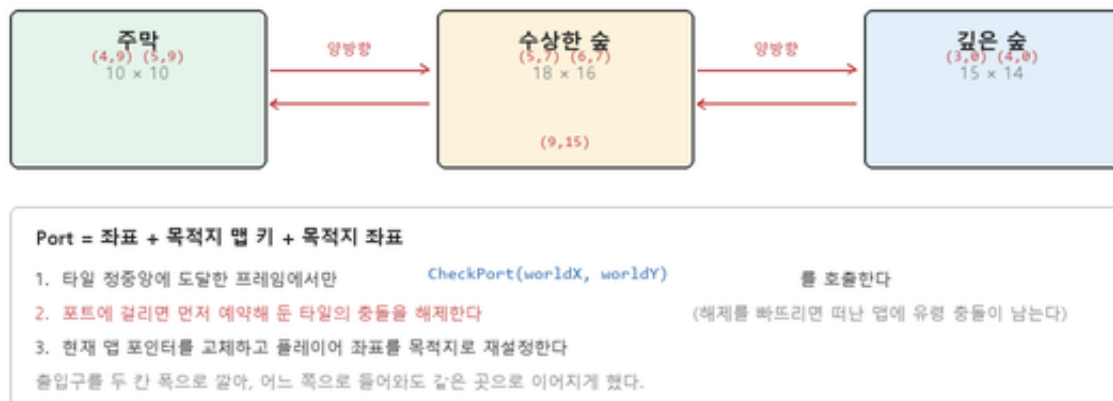
정리

- 판정 비용이 $O(1)$ 이다. 몬스터가 몇 마리든 갈 곳의 타일 하나만 조회한다.
- 벽 · 캐릭터 · NPC 를 같은 방식으로 다룬다. 벽은 처음부터 충돌인 타일, 캐릭터는 매 프레임 자기 타일에 충돌을 기록하는 존재일 뿐이다.
- 몬스터의 배회 로직도 이 정보를 그대로 쓴다. 네 방향 타일을 조회해 지나갈 수 있는 방향만 후보로 모으므로, 벽으로 돌진하는 움직임이 애초에 나오지 않는다.

6. Port 를 통한 맵 전환

맵은 각자 크기 · 타일 배열 · 몬스터 · NPC · 장식물 목록을 갖는 독립된 객체이고, `MapManager` 가 이름을 키로 들고 있다가 현재 맵 포인트만 갈아끼운다. 맵 사이를 잇는 통로는 `Port` 라는 오브젝트로 표현했다 — 좌표 하나에 목적지 맵 이름과 목적지 좌표를 붙여둔 것이다.

그림 4. Port 오브젝트를 통한 맵 전환



6.1 전환 처리

포트 판정은 타일 정중앙에 도달한 프레임에서만 일어난다. 이동 중에 걸쳐 있는 상태로 판정하면 한 번의 이동으로 두 번 전환되는 문제가 생긴다.


```

Port* port = currentMap->CheckPort (_worldX, _worldY);

if (port)
{
    // 맵이 이동 되므로 예약 지점 충돌을 해제한다
    currentMap->GetTile (_worldX, _worldY)->SetCollider (Collider::NORMAL);

    MapManager::GetInst ()->SetCurrentMap (port->GetDestinationMapKey ());
    Player::GetInst ()->SetWorldX (port->GetDestinationWorldX ());
    Player::GetInst ()->SetWorldY (port->GetDestinationWorldY ());
}

```

출입구는 **두 칸** 폭으로 깔아 두 좌표에 각각 포트를 두고 같은 목적지로 보냈다. 한 칸짜리 문은 정확히 그 칸을 밟아야 해서 조작이 답답해지기 때문이다. 반대편 맵에도 대응하는 포트를 뒤서 양방향으로 오갈 수 있다.

6.2 맵 구성

맵은 기본 타일로 격자를 채운 뒤 필요한 칸만 덮어쓰는 방식으로 만들었다. 벽은 `SetTile` 에 충돌 속성을 함께 넘기고, 주막 건물처럼 **들어갈 수 없는 구조물**은 3×5 영역을 충돌로 채운 다음 출입구 두 칸만 다시 통행 가능으로 되돌려 표현했다.

```

map->CreateMap (18, 16, "Tile 4"); // 기본 타일로 채우기

for (int i = 0; i < 5; i++) // 건물 영역을 막고
{
    map->SetTile (3 + i, 5, ..., Object::COLLISION);
    map->SetTile (3 + i, 6, ..., Object::COLLISION);
    map->SetTile (3 + i, 7, ..., Object::COLLISION);
}
map->SetTile (5, 7, ..., Object::NORMAL); // 출입구 두 칸만 열기
map->SetTile (6, 7, ..., Object::NORMAL);

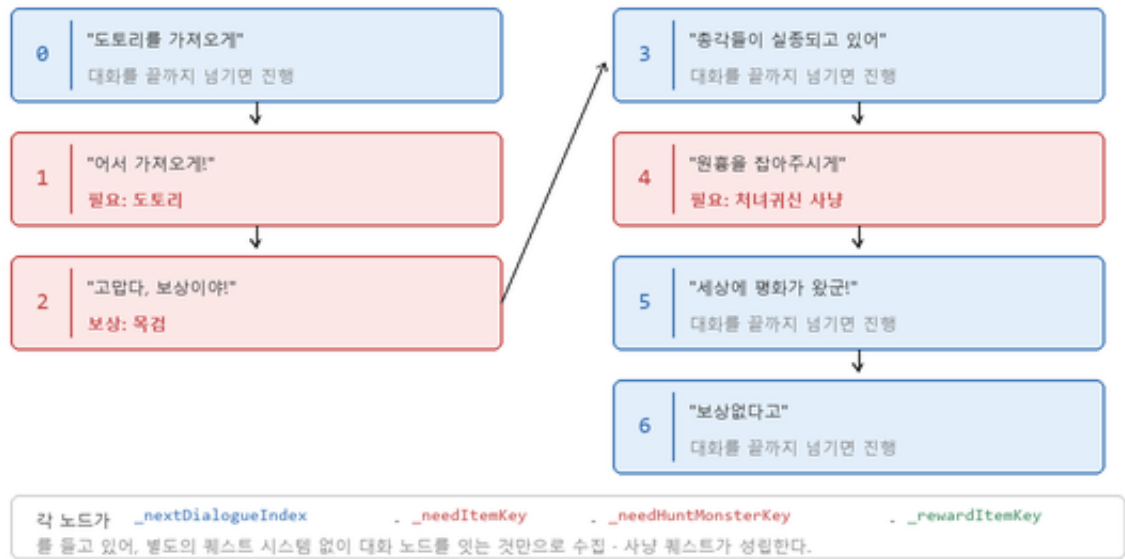
for (int i = 0; i < 14; i++) // 몬스터 무작위 배치
    map->CreateMonster ("다람쥐", rand () % 18, rand () % 16);

```

7. 대화 노드 그래프로 구현한 퀘스트

퀘스트 시스템을 따로 만들지 않고, **대화 그 자체를 상태로** 다뤘다. NPC 는 대화 목록과 '지금 몇 번째 대화인지' 를 가리키는 인덱스를 들고 있고, 각 대화 노드는 다음에 갈 노드 번호와 **진행 조건**을 갖는다. 노드를 잇는 것만으로 수집 퀘스트와 사냥 퀘스트가 성립한다.

그림 5. 대화 노드 그래프로 구현한 퀘스트 진행



7.1 노드가 들고 있는 정보

<code>_nextDialogueIndex</code>	이 대화가 끝나면 NPC 를 몇 번 노드로 옮길지
<code>_needItemKey</code>	가지고 있어야 넘어갈 수 있는 아이템 — 확인되면 인벤토리에서 소모된다
<code>_needHuntMonsterKey</code>	잡아야 넘어갈 수 있는 몬스터 — 플레이어의 사냥 기록을 조회한다
<code>_rewardItemKey</code>	대화를 완료하면 지급하는 아이템
<code>_kind</code>	대화(TALK) · 선택(SELECTION) · 상점(SHOP) 중 어떤 창으로 그릴지

7.2 진행 판정

대화를 열 때마다 먼저 조건을 검사한다. 조건이 걸린 노드는 **조건이 충족되는 순간 자기를 건너뛰고** 다음 노드를 대신 띄운다. 덕분에 "아직 못 가져왔군" 과 "고맙다, 보상이야" 를 분기문 없이 두 개의 노드로 자연스럽게 표현할 수 있다.

```
if (_needItemKey != "") // 필요아이템이 있는 경우
{
    for (인벤토리의 각 아이템)
    {
        if (_needItemKey == 아이템 이름)
        {
            인벤토리에서 제거; // 퀘스트 아이템 소모
            npc->SetCurrentDialogue (_nextDialogueIndex);
            npc->GetCurrentDialogue ()->SetEnabled (true); // 다음 노드로 교체
            _enabled = false;
            return;
        }
    }
}
```

사냥 조건은 같은 방식으로 `Player` 가 들고 있는 사냥 기록 목록을 조회한다. 몬스터가 죽을 때 자기 이름을 그 목록에 넣어두기 때문에, 퀘스트 쪽에서는 별도의 이벤트 연결 없이 "처녀귀신을 잡았는가" 를 물어보기만 하면 된다.

7.3 진행도를 어디에 두는가

NPC 는 맵마다 별도의 인스턴스로 배치되지만, 퀘스트 진행도는 **한 곳에만** 있어야 한다. 그래서 대화 상태는 맵에 놓인 인스턴스가 아니라 **NpcManager** 가 들고 있는 **원본 데이터**에 저장하고, 맵의 인스턴스는 이름으로 원본을 찾아가 대화를 갱신 · 출력한다. 같은 NPC 를 여러 맵에 배치해도 진행도가 갈라지지 않는다.

```
void Npc::Update ()
{
    // 자기 대화가 아니라, 이름으로 찾은 원본의 대화를 갱신한다
    NpcManager::GetInst ()->GetNpcData (_name)->GetCurrentDialogue ()->Update ();
}
```



'사냥터 안내인' 과의 대화. 노트 0번의 첫 페이지가 출력된 상태.

8. 상태 머신과 몬스터 행동

플레이어와 몬스터는 모두 **switch** 기반 유한 상태 머신으로 동작한다. 플레이어는 **IDLE · MOVE · ATTACK · FARMING · OTHER_ACTION** 다섯 상태를, 몬스터는 **IDLE · MOVE · ANGRY · ATTACK** 네 상태를 갖는다.

그림 6. 몬스터 상태 머신



- IDLE 은 네 방향 중 지나갈 수 있는 타일만 골라 무작위로 하나 잡는다 — 벽에 처박히는 움직임이 나오지 않는다.
- ANGRY 는 플레이어 방향을 구해 부호만 남기고, 대각선이 나오면 무효로 돌린다 (타일은 4방향만 허용).

8.1 배회 — 갈 수 있는 방향만 후보로

몬스터는 2초마다 방향을 다시 뽑는다. 이때 네 방향을 무작위로 고르고 막혔으면 포기하는 방식이 아니라, **먼저 지날 수 있는 방향만 모아** 그 안에서 뽑는다. 벽에 부딪혀 제자리에서 버벅이는 움직임이 나오지 않고, 후보가 없으면 아예 판단을 건너뛰는 것이다.

8.2 추격 — 대각선을 걸러내기

피격당하면 **ANGRY** 로 전환해 플레이어를 향한다. 방향은 좌표 차를 구해 부호만 남기는 방식으로 정규화하는데, 이러면 대각선 방향이 나올 수 있다. 타일맵에서는 4방향만 허용되므로 **대각선이 나온 경우는 무효로 돌려 배회** 상태로 되돌린다.

```
_dirX = Player::GetInst ()->GetWorldX () - _worldX;
_dirY = Player::GetInst ()->GetWorldY () - _worldY;

if (_dirX != 0) _dirX /= abs (_dirX); // 부호만 남긴다
if (_dirY != 0) _dirY /= abs (_dirY);

// 대각선은 무효로 한다. 타일맵에서 적은 0도, 90도, 180도, 270도로만 회전 가능하기 때문
if (_dirX != 0 && _dirY != 0) { _state = IDLE; break; }
```

8.3 공격 — 애니메이션이 타이밍을 정한다

몬스터는 플레이어를 마주한 채 1.5초가 지나면 공격에 들어간다. 타격 판정은 공격 애니메이션이 **끝나는 프레임**에서 이뤄지고, 그 순간에도 플레이어가 앞에 있는지 다시 확인한다. 선딜 중에 피해 범위를 벗어나면 맞지 않는, 원작과 같은 감각이 나온다.

8.4 상태와 방향으로 애니메이션을 자동 선택

캐릭터는 (상태 × 방향) 조합에 대응하는 16개의 애니메이션 슬롯을 갖는다. 애니메이션을 코드 곳곳에서 직접 바꾸면 상태와 화면이 어긋나기 쉬우므로, **매 프레임 끝에 현재 상태와 방향으로부터 슬롯을 결정**하도록 한 곳에 모았다. 이동 · 공격 · 채집 로직은 애니메이션을 전혀 신경 쓰지 않는다.

```
void Character::End ()
{
    switch (_state)
    {
        case MOVE:
            if (_dirX == 0 && _dirY == 1) _animation = FRONT_WALK;
            if (_dirX == 0 && _dirY == -1) _animation = BACK_WALK;
            if (_dirX == -1 && _dirY == 0) _animation = LEFT_WALK;
            if (_dirX == 1 && _dirY == 0) _animation = RIGHT_WALK;
            break;
        ...
    }
}
```

9. 전체 구조와 데이터 관리

9.1 프로토타입 패턴으로 원본과 인스턴스를 분리

몬스터 · 아이템 · NPC · 애니메이션은 모두 같은 방식으로 관리한다. 매니저가 **문자열 키로 원본 한 벌**을 들고 있고, 맵에 배치할 때는 원본을 **복사 생성**해 인스턴스를 만든다. 다람쥐 14마리를 배치해도 스탯 · 애니메이션

정의는 한 곳에만 존재한다.

```
void Map::CreateMonster (string monsterKey, int wx, int wy)
{
    // 매니저가 들고 있는 원본을 복사해서 인스턴스를 만든다
    Monster* monster = new Monster (*MonsterManager::GetInst ()->GetMonsterData (monsterKey));
    monster->SetWorldX (wx);
    monster->SetWorldY (wy);
    _vMonsters.push_back (monster);
}
```

9.2 다중 독립 타이머

게임 곳곳에 서로 다른 주기가 필요하다 — 입력 연타 억제(100ms), 몬스터 방향 재추첨(2000ms), 공격 선딜(1500ms), 대화 넘김 디바운스(500ms), 상점 커서 이동(200ms). 이것 개별 변수로 두면 관리가 흩어지므로, **타이머 배열을 가진 믹스인 클래스**를 만들어 필요한 클래스가 상속받게 했다. 한 오브젝트가 인덱스로 구분되는 여러 개의 독립 타이머를 동시에 굴릴 수 있다.

```
CreateTime (4); // 이 오브젝트는 독립 타이머 4개를 쓴다

FlowTime (0); // 0번 타이머에 경과 시간 누적

if (IsTime (0, 500)) // 500ms 지났는가 (참이면 자동으로 0으로 리셋)
{
    ++_page; // 대화 다음 페이지
}
```

IsTime 이 참을 반환할 때 스스로 리셋하도록 만든 게 핵심이다. 호출부에서 리셋을 잊어 입력이 한 번만 먹거나 계속 먹는 실수가 구조적으로 막힌다.

9.3 클래스 구조

Object	타일 · 픽셀 좌표, 방향, 충돌 속성, 밟은 타일 갱신 — 화면에 존재하는 모든 것의 기반
Character	Object 상속. 상태 머신, 스탯, 16슬롯 애니메이션 선택 — Player · Monster 의 공통 기반
Player / Monster / Npc	각자의 상태 머신과 행동. Npc 는 대화 노드 목록을 갖는다
Tile / Port / Item / Ornament	맵을 구성하는 요소들. 모두 Object 를 상속해 같은 방식으로 그려진다
Map	타일 배열과 배치된 오브젝트 목록. 자기 안의 모든 것을 갱신 · 렌더링한다
RenderManager	이미지 등록 · 백버퍼 관리 · 스프라이트와 텍스트 출력
~Manager (6종)	원본 데이터 보관과 조회. 모두 Singleton 템플릿 기반
Time	다중 독립 타이머 믹스인

9.4 메인 루프

구조는 단순하다. 렌더와 업데이트를 번갈아 호출하고, 갱신은 플레이어 → 현재 맵(타일 · 아이템 · 몬스터 · NPC · 장식물) → UI 순으로 전파된다. 맵이 자기 안의 오브젝트를 책임지므로, 맵을 갈아끼우면 갱신 대상도 함께 바뀐다.

```
while (true)
{
    mainGame.Render (); // Begin → 맵 → UI → 대화창 → End (백버퍼 전송)
    mainGame.Update (); // 플레이어 → 현재 맵 → UI
}
```

10. 결과와 회고

결과

- 게임 엔진 없이 Win32 API 와 GDI 만으로 2D MMORPG 의 핵심 루프를 6일 안에 완성했다.
- 텍스트만 출력하는 콘솔 환경에서 원작에 가까운 그래픽 화면을 재현했다.
- 타일맵 · 이동 · 충돌 · 전투 · 인벤토리 · 상점 · 다단계 퀘스트가 모두 동작한다.
- 리소스까지 원작 화면에서 직접 추출해 정리했다.

돌아보며

가장 값진 경험은 **환경의 제약을 우회하는 방법을 스스로 찾아낸 것이었다**. "콘솔에서는 그림을 못 그린다" 를 전제로 받아들이지 않고, 콘솔 창도 윈도우라는 사실에서 출발해 창 핸들 → DC → GDI 라는 경로를 찾아낸 과정이 이 작품의 출발점이자 핵심이다.

설계 측면에서는 **판정 지점을 한 곳으로 모으는 것**의 효과를 체감했다. '타일 정중앙에 도달한 프레임' 하나에 맵 이동 · 입력 재판정 · 상태 전환을 모두 묶어두니, 이동 중 애매한 상태에서 생기는 버그가 애초에 발생하지 않았다. 충돌을 타일에 기록하는 방식도 마찬가지로, 벽과 캐릭터와 NPC 를 같은 규칙으로 다룰 수 있게 해줬다.

반면 지금 다시 만든다면 고칠 부분도 분명하다. 맵 · 몬스터 · 퀘스트 데이터가 모두 **코드에 하드코딩**되어 있어 내용을 바꿀 때마다 재컴파일이 필요했다. 외부 데이터 파일로 분리했다면 훨씬 빠르게 반복할 수 있었을 것이다. 프레임 제한이 없어 이동 속도가 실행 환경의 성능에 좌우되는 것도 당시에는 미처 신경 쓰지 못한 부분이다.

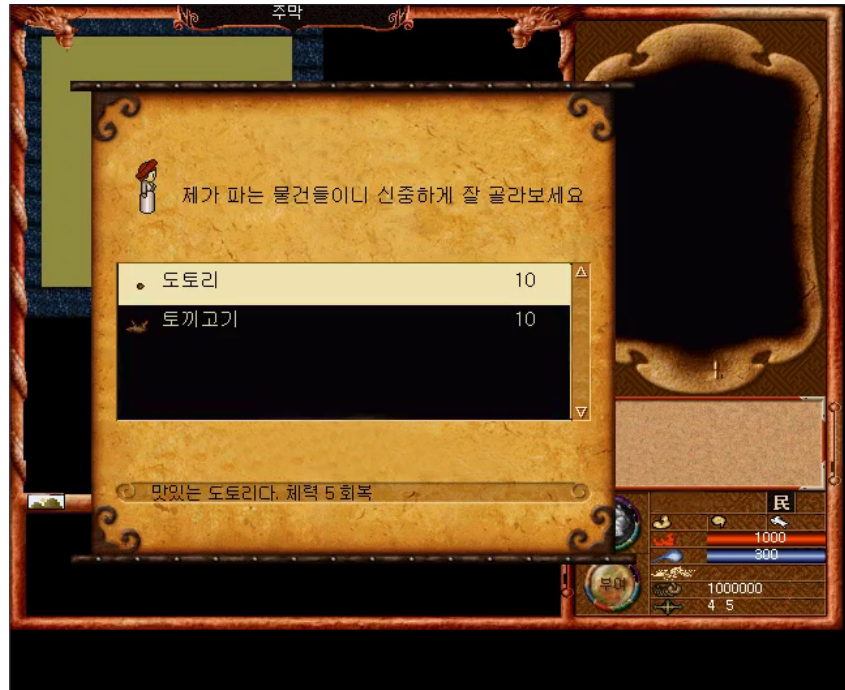
빌드 및 실행

원본은 Visual Studio 2015(v140) · Windows SDK 8.1 프로젝트다. 최신 환경에서 빌드하려면 플랫폼 툴셋과 SDK 버전만 현재 설치본으로 바꾸면 되고, 소스 수정은 필요하지 않다. 실행 시에는 실행 파일과 같은 위치에 **Image** 폴더와 **bgm.wav** 가 있어야 한다.

Windows 11 에서 실행할 때

Windows 11 의 기본 콘솔 호스트(Windows Terminal)는 콘솔 창에 GDI 로 직접 그리는 방식을 지원하지 않아 화면이 검게만 나온다. 레거시 **conhost.exe** 를 통해 실행하면 정상적으로 출력된다.

```
conhost.exe 161225_Baram.exe
```



주막 NPC 의 상점. 아이템 목록 · 가격 · 설명이 표시되고 커서로 선택해 구매한다.